– Training Regression Models –
LASSO

Anca Ralescu
CS Department
University of Cincinnati
Cincinnati, OH, USA
Anca.Ralescu@uc.edu

The Mean Absolute Error (MAE) is defined as

$$MAE(\mathbf{X}, h) = \frac{1}{m}\sum_{i=1}^{m} \|h(\mathbf{x}^{(i)}) - y^{(i)}\|_1$$

where $\|\cdot\|_1 = |\cdot|$ is the l_1 norm;

Lasso Regression: Least Absolute Shrinkage and Selection Operator Regression

Cost function

$$J(\mathbf{w}) = MSE(\mathbf{w}) + \alpha \sum_{\mathbf{i=1}}^{n} |\mathbf{w}_i|$$

Taking into account that

$$|x| = \begin{cases} -x & \text{if} \quad x < 0 \\ 0 & \text{if} \quad x = 0 \\ +x & \text{if} \quad x > 0 \end{cases}$$

We can rewrite the LASSO cost function as

$$J(\mathbf{w}) = MSE(\mathbf{w}) - \alpha \sum_{\mathbf{i=1},\mathbf{w}_i<0}^{n} \mathbf{w}_i + \alpha \sum_{\mathbf{i=1},\mathbf{w}_i=0}^{n} \mathbf{w}_i + \alpha \sum_{\mathbf{i=1},\mathbf{w}_i>0}^{n} \mathbf{w}_i$$

which means

$$J(\mathbf{w}) = MSE(\mathbf{w}) - \alpha \sum_{\mathbf{i=1},\mathbf{w}_i<0}^{n} \mathbf{w}_i + \alpha \cdot 0 + \alpha \sum_{\mathbf{i=1},\mathbf{w}_i>0}^{n} \mathbf{w}_i$$

The Lasso cost function is not differentiable at $\mathbf{w}_i = 0$ (for $i = 1, 2, \ldots, n$). Gradient Descent still works fine if you use a *subgradient* vector g instead when any $\mathbf{w}_i = 0$.

$$g(\mathbf{w}, J) = \nabla_{\mathbf{w}} MSE(\mathbf{w}) + \alpha \cdot sign(\mathbf{w})$$

where $sign$ is defined as

$$sign(x) = \begin{cases} -1 & \text{if} \quad x < 0 \\ 0 & \text{if} \quad x = 0 \\ +1 & \text{if} \quad x > 0 \end{cases}$$

Thus

$$sign(\mathbf{w}) = \begin{pmatrix} sign(\mathbf{w}_0) \\ sign(\mathbf{w}_1) \\ \cdots \\ sign(\mathbf{w}_n) \end{pmatrix}$$

Figure 1 illustrates Lasso with various levels of regularization - left linear; right polynomial.

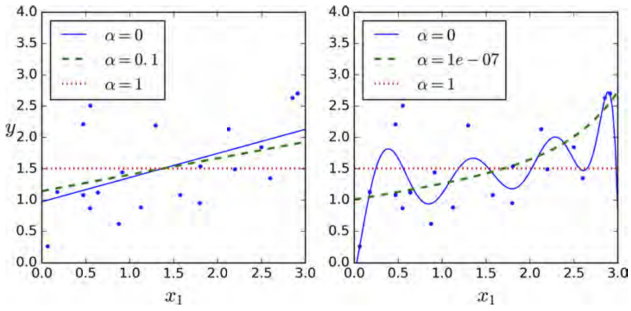


Figure 1: Lasso Regression.

Lasso Regression tends to completely eliminate the weights of the least important features (i.e., set them to zero).

For example, the dashed line in the right plot in Figure 1 ($\alpha = 10^{-7}$) looks quadratic, almost linear: all the weights for the high-degree polynomial features are equal to zero.

That is, Lasso Regression automatically performs feature selection and outputs a sparse model (i.e., with few nonzero feature weights).

Elastic Net

Middle ground between Ridge Regression and Lasso Regression

$$J_{elastic\,net}(\mathbf{w}; \alpha, r) = MSE(\mathbf{w}) + r\alpha \sum_{\mathbf{i=1}}^{n} |\mathbf{w}_i| + \frac{1-r}{2} \alpha \sum_{\mathbf{i=1}}^{n} \mathbf{w}_i^2,$$

where α and r are hyperparameters. Then

$$\nabla J_{elastic\,net}(\mathbf{w}; \alpha, r) = \nabla MSE(\mathbf{w}) + r\alpha \cdot sign(\mathbf{w}) + (1-r)\alpha \mathbf{w}$$

Choosing the regression

- Avoid plain regression
- Prefer Lasso or Elastic Net when one suspects that only some features are useful
- Prefer Elastic to Lasso when $n > m$ or when features are strongly correlated

Early Stop ("beautiful free lunch")

Stop when the validation error reaches a minimum. Figure 2 shows a complex model (in this case a high-degree Polynomial Regression model) being trained using Batch Gradient Descent.

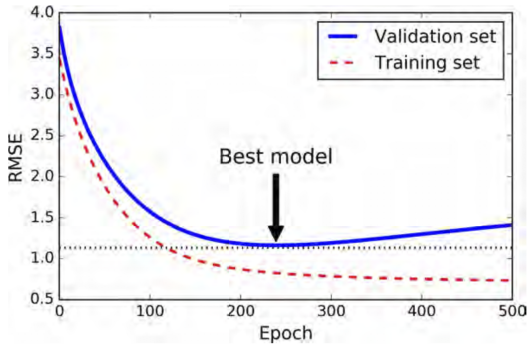


Figure 2: Early stopping regularization.

As the algorithm learns and its prediction error (RMSE) on the training set naturally goes down, and so does its prediction error on the validation set.

However, after a while the validation error stops decreasing and actually starts to go back up: the model has started to overfit the training data.

Logistic Regression

Used to estimate probabilities.

$$\widehat{p} = h_{\mathbf{w}}(\mathbf{x}) = \sigma(\mathbf{x}^T\mathbf{w}),$$

where

$$\sigma(t) = \frac{1}{1 + exp^{(-t)}}$$

shown in Fig 3.

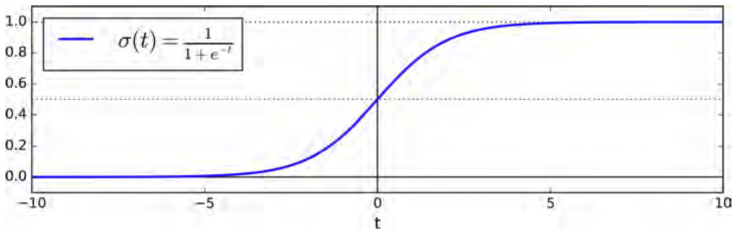


Figure 3: The logistic function.

The model prediction is based on $\widehat{p}$

$$\widehat{y} = \begin{cases} 0 & \text{if} \quad \widehat{p} < 0.5 \\ 1 & \text{if} \quad \widehat{p} \geq 0.5 \end{cases}$$

therefore $\widehat{y} = 1$ when $\mathbf{x}^T\mathbf{w} \geq 0$ and 0 if $\mathbf{x}^T\mathbf{w} < 0$.

Training the logistic regression model

The cost function for a single instance is given by

$$c(\mathbf{w}) = \begin{cases} -\log \widehat{p} & \text{if } y = 1 \\ -\log(1 - \widehat{p}) & \text{if } y = 0 \end{cases}$$

We can write $c(\mathbf{w})$ in one equation as follows:

$$c(\mathbf{w}) = -y \log \widehat{p} + (1 - y) \log(1 - \widehat{p})$$

The cost function over all m training instances is then

$$J(\mathbf{w}) = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log \widehat{p}^{(i)} + (1 - y^{(i)}) \log(1 - \widehat{p}^{(i)}) \right]$$

No closed form solution (no normal equation).

Apply Gradient Descent, which works because $J(\mathbf{w})$ is convex.

$$\frac{\partial}{\partial \mathbf{w}_j} J(\mathbf{w}) = \sum_{i=1}^{m} \left(\sigma(\mathbf{w}^T\mathbf{x}^{(i)}) - y^{(i)} \right) x_j^{(i)}$$

Once we compute $\nabla J(\mathbf{w})$ we can apply Batch/Stochastic/MiniB Gradient Descent.

Decision Boundaries

Use the Iris Data set shown in Figure 4

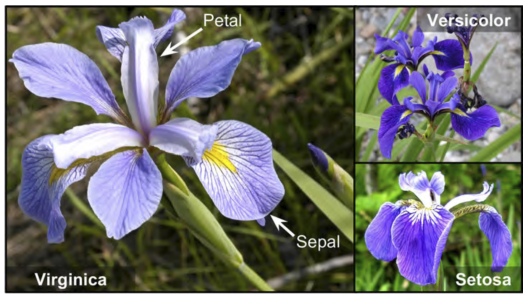


Figure 4: Flowers of three iris flowers.

Training a logistic regression model on the petal width, yields the decision boundary shown Fig. 5.

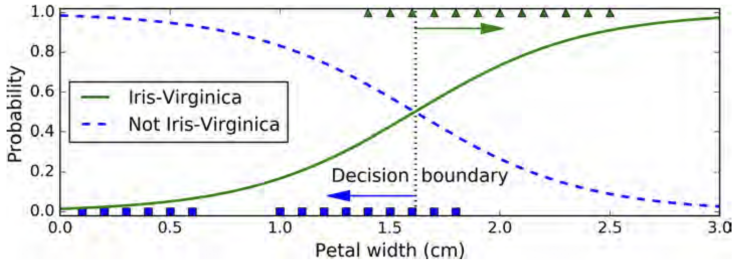


Figure 5: Estimated probabilities and decision boundary based on the petal width.

- The petal width of *Iris-Virginica flowers* are represented by triangles: its range is $[1.4, 2.5]$ cm.
- Other iris flowers petal width are represented by squares: generally, they have a smaller petal width: $[0.1, 1.8]$ cm
- Overlap: $\approx$ 2cm
- Above about 2 cm the classifier is highly confident that the flower is an Iris-Virginica
- Below 1 cm it is highly confident that it is not an Iris-Virginica
- In between these extremes, the classifier is unsure:
  - it predicts the class – whichever class is the most likely.

- Therefore, there is a decision boundary at around 1.6 cm where both probabilities are equal to 0.5:
  - if the petal width is higher than 1.6 cm, the classifier will predict that the flower is an Iris- Virginica,
  - else it will predict that it is not (even if it is not very confident)

Figure 6 shows the same data in the petalLength - petalWidth space. Once trained, the

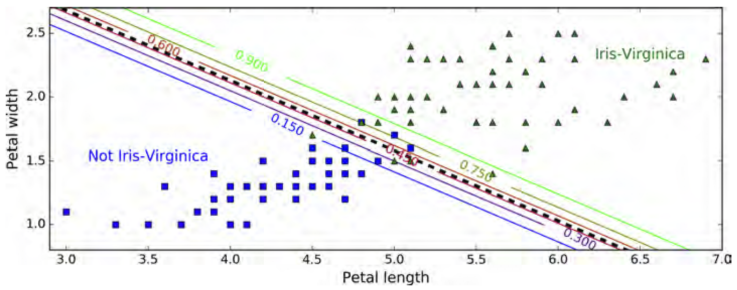


Figure 6: Linear Decision Boundary: dashed line estimates 0.5 probability; bottom left, 0.15, top right 0.90.

Logistic Regression classifier can estimate the probability that a new flower is an *Iris virginica*. Logistic regression can be regularized.

Softmax Regression / Multinomial Logistic Regression

Generlize the Logistic Regression to support multiple classes directly.

When given an instance x, the Softmax Regression model first computes a score $s_k(x)$ for each class k, then estimates the probability of each class by applying the *softmax* function (also called the normalized exponential) to the scores.

$$s_k(\mathbf{x}) = \mathbf{x}^T \mathbf{w}^{(k)}$$

where $\mathbf{w}^{(k)}$ is the parameter vector for class k.
The matrix $\mathbf{w}$ holds all the vectors $\mathbf{w}^{(k)}$.
The softmax function for class k:

$$\widehat{p}_k = \sigma(s(\mathbf{x}))_k = \frac{exp(s_k(\mathbf{x}))}{\sum_{j=1}^{K} exp(s_j(\mathbf{x}))},$$

where

- K is the number of classes
- $\mathbf{s}(\mathbf{x})$ is a vector containing the scores for each class for instance $\mathbf{x}$
- $\sigma(\mathbf{s}(\mathbf{x}))_k$ is the estimated probability that $\mathbf{x}$ belongs to class k, given the scores for each class

The softmax regression classifier prediction is

$$\widehat{y} = \arg\max_k \sigma(\mathbf{s}(\mathbf{x}))_k = \arg\max_k s_k(\mathbf{x}) = \arg\max_k \left(\left(\mathbf{w}^{(k)} \right)^T \cdot \mathbf{x} \right)$$

Training the Softmax

Use the cross entropy

$$J(\mathbf{w}) = -\frac{1}{m} \sum_{i=1}^{m} \sum_{k=1}^{K} y_k^{(i)} \log \left(\widehat{p}_k^{(j)} \right),$$

where

- $y_k^{(i)}$ is equal to 1 if the target class for the ith instance is k; otherwise, it is equal to 0.

The gradient for the cross entropy for class k is given by

$$\nabla_{\mathbf{w}^{(k)}} J(\mathbf{w}) = \frac{1}{m} \sum_{i=1}^{m} \left(\widehat{p}_k^{(i)} - y_k^{(i)} \right) \mathbf{x}^{(i)}$$